

Phase 7: RISC-V Cache

EEL 4768 Computer Architecture: Spring 2026

Project Introduction

In this project phase, you will be optimizing your multi-cycle RISC-V 32RI processor by optionally adding additional modules to decrease its CPI. A top module skeleton will be provided which you **must use** or the autograder may not be able to process your files.

Your processor should **only use instructions found in the provided RV32I reference card posted in webcourses**. Using any instructions outside of the base RV32I instruction set will result in **a zero** for the specific problem.

Submission Requirements

You are permitted to use generative artificial intelligence (e.g. ChatGPT); if you choose to do so, indicate where you use AI in your submission, and why you used it. Please ensure your usage of LLMs adheres to the course policies.

Submit the Verilog files as individual, unzipped files directly to Gradescope. In addition to the Verilog files, please include a word document or pdf detailing the contributions of you and your team members, along your use of AI.

- RISCV_TOP.v
- Additional Verilog files
- assembler.(py, cpp)
- group_contributions.pdf

In Gradescope, after submitting you can select your group for that submission. One submission is needed, but if you fail to select your group members in that submission, they may receive a zero for this phase.

Environment setup

You may use any simulator as long as it's capable of simulating Verilog. If you want to use Vivado like how you used it in digital systems, you are more than welcome to.

Another simulator that is recommended to be used is [Verilator](#), which compiles the simulation of the Verilog files with a C++ wrapper. For assistance with installation, please come to my office hours.

[Icarus](#) is another lightweight option.

Your assembler can be writing in one of two languages of your choice.

- C++
- Python

1. Branch Predictor (2 pts)

Implement branch prediction hardware that reduces the penalty cycles from improper speculative instruction processing.

2. Multiplication (1 pt)

Introduce a dedicated multiplication unit to the ALU. Additionally, you will have to change your assembler to be capable of decoding the mult/multi instruction.

Instruction	Type	Opcode	Funct3	Funct7
mult	R	011 0011	000	000 0001
multi	I	001 0011	000	000 0001

3. SIMD (2 pts)

Finally, you can design a SIMD array for vector processing, which can compute multiple multiply and accumulate instructions in parallel. Your SIMD array should be a vector with 4 multiply-accumulate units and will also need a special instruction.

Instruction	Type	Opcode	Funct3	Funct7
mac4	R	011 0011	000	000 0010
mac4i	I	001 0011	000	000 0010